

The Living Map Technical Report

IEEE RAS x AESS Tunisia, TSYN14 Technical Challenge, Phase 1 Environment chosen: Urban Search and Rescue

1. Overview

The Living Map gives a GPS-denied, disconnected space its own memory. A Writer robot (a hovering drone) explores a collapsed-building interior autonomously, senses hazards and victims, and deposits small radio beacons at each finding. An Outside Network Area - the only bridge between the disconnected zone and the outside world - receives the Writer's beacon log at the entrance, translates its private coordinates into real-world GPS, carries the data to a distant Command Post, and briefs an Executor robot (a ground rover) before it enters. The Executor then navigates purely by inherited beacon knowledge, trusting fresh beacons and re-verifying stale ones with its own sensing before acting.

No direct link exists between either robot and the Command Post anywhere in the system - every byte that crosses the inside/outside boundary passes through the Outside Network Area's four roles: RECEIVE, TRANSLATE, CARRY, BRIEF.

2. System Architecture

[Writer drone]	[Outside Network Area]	[Command Post]
explores (sweep)	1. RECEIVE <-- entrance only	live map
senses events --beacon-->	2. TRANSLATE (xy -> GPS) -->	(auto-
drops beacons	3. CARRY ----->	refreshing
returns to entrance	4. BRIEF --mission-->	[Executor rover] HTML)
		navigates to
		beacons, acts,
		reports back
		(via Outside
		Network only)

The implementation is a 4-package ROS2 (Humble) workspace driving a Gazebo Classic simulation:

Package	Responsibility
living_map_description	Robot models: Writer drone and Executor rover
living_map_gazebo	The USAR world and spawn launch
living_map_system	The four behavioural nodes and the Beacon schema
living_map_bringup	Top-level launch composing world + robots + nodes

3. Writer Robot Autonomy

The Writer is modelled as a quadrotor whose body has gravity disabled in Gazebo and is driven by libgazebo_ros_planar_move, giving it horizontal velocity and yaw control while holding a constant altitude - a kinematic hover simplification, in keeping with the challenge's focus on the data and decision architecture rather than flight-dynamics modelling.

Exploration uses a deterministic boustrophedon sweep over the building's footprint: a fixed list of waypoints spanning the interior at regular intervals, each one reached by simple proportional heading control (angular.z proportional to bearing error, linear.x capped and reduced near the goal). This was chosen over a purely reactive/random-wander strategy specifically because it guarantees coverage - a hard requirement for a robot that gets one exploration pass before its results are locked in and handed off. A real deployment would replace this with frontier-based exploration driven by the lidar/SLAM map, which is a drop-in upgrade at the EXPLORE state's waypoint-selection logic - the rest of the pipeline (detection, beacon drop, handoff) is unchanged by that upgrade.

The Writer's control loop is a 4-state machine: EXPLORE -> RETURN -> TRANSMIT -> DONE. It transitions out of EXPLORE when the sweep completes, a time budget expires, or three events have been found - whichever comes first, modelling a real mission-time/battery constraint.

4. Event Detection and Beacon Deposition

Detection is proximity-based against `/gazebo/model_states`: when the Writer's position comes within 1.3m of an undetected event marker in the world, it is classified (hazard or victim by the marker's name - in a physical build this would be the output of a thermal/gas/vision classifier) and a Beacon is constructed and appended to the Writer's local log.

Deposition is modelled by spawning a small physical marker model in Gazebo at the event's location via the `/spawn_entity` service - the beacon is a real object left behind in the world, not just a log entry, matching the challenge's "small radio beacons" framing. In a physical build this step is the beacon-ejection mechanism firing.

5. Beacon Message and Signal Design

Each beacon carries exactly the fields the challenge specifies, JSON-encoded as a transparent stand-in for a packed RF payload (`beacon_message.py`):

Field	Meaning
id	which beacon is speaking
what	hazard / victim / junction / exit
x, y	local writer-frame coordinates (the WHERE)
distance	metres from the entrance/previous beacon
when	unix timestamp - this is what makes it age
version	lets a later agent overwrite a beacon's record

Aging is derived from when, not stored redundantly: fresh (under 5 min), aging (under 15 min), stale (beyond). The Executor consults this at the moment it reaches each target, not at mission-briefing time, so trust reflects the beacon's age when acted on, not when it was reported - an Executor that takes a long route to a far beacon correctly treats it with more suspicion than one it reaches immediately.

A real RF implementation would pack this into a handful of bytes (id: 1B, what: 1B enum, x/y: 2B each fixed-point, distance: 1B, when: 4B epoch, version: 1B - comfortably under 16 bytes) for a LoRa-class broadcast; the JSON encoding here is purely to keep the simulation's data flow inspectable on the ROS bus.

6. Frame Translation

`outside_network_node`'s `local_to_gps()` anchors the entrance at a fixed, pre-surveyed real-world GPS reference point and converts the Writer's local (x, y) metres into latitude/longitude via an equirectangular (flat-earth) approximation - accurate to well under a metre at the scale of a single building, which is the relevant scale here. Swapping in a full geodesic projection for larger sites is a one-function change; no other node depends on the projection's internals.

7. Outside Network Area

Implemented as a single node (`outside_network_node`) with exactly the four roles the challenge specifies, each logged explicitly so the data flow is auditable:

1. RECEIVE - the only subscriber to `/writer/to_outside_network` and `/executor/to_outside_network`. These are the sole inbound channels from the disconnected zone.
2. TRANSLATE - local (x, y) to GPS, as above, plus a computed freshness label per beacon.
3. CARRY - forwards the translated data to `/command_post/beacons`, documented here as the simulation's stand-in for a satellite/LoRa uplink (per the challenge's own Q&A guidance that WiFi/MQTT-over-internet is an acceptable stand-in for the real-world long-range link, which should be named explicitly as the intended real-world equivalent - done above).
4. BRIEF - builds the Executor's mission (one target per beacon, with an action derived from its type: hazard to seal, victim to extract) and publishes it to `/outside_network/executor_brief` - the Executor's only inbound channel, consulted exactly once, before it re-enters the space.

The Command Post, symmetrically, has no publishers aimed at either robot - structurally enforcing "no direct link between the robots and the command post" rather than relying on convention.

8. Executor Robot

On receiving its one-time briefing, the Executor sorts targets by distance from the entrance and visits

them in order using the same proportional go-to-goal controller as the Writer. At each target it checks the beacon's freshness: fresh beacons are acted on immediately; a stale beacon triggers a 2-second verify pause during which the Executor checks its own sensing (proximity to the real marker, standing in for a physical sensor check) before deciding whether to trust it or skip it in favour of the robot's own observations - directly implementing "trusts by age" from the challenge brief. Once every target has been attempted, the Executor reports mission completion back through the Outside Network Area (never directly to the Command Post), closing the loop symmetrically with the Writer's handoff.

9. Data Flow Summary

```
event (ground truth in world)
-> Writer proximity sense -> Beacon record (id, what, x, y, distance, when=now, version)
-> Writer local beacon log
-> (at entrance) /writer/to_outside_network [RECEIVE]
-> local_to_gps(x,y) -> lat, lon, freshness [TRANSLATE]
-> /command_post/beacons [CARRY] -> live_map.html render
-> /outside_network/executor_brief (mission_id, targets) [BRIEF]
-> Executor sorts by distance, navigates, checks freshness at arrival,
    verifies-or-trusts, acts, logs completion
-> /executor/to_outside_network (status complete, completed list)
-> /command_post/mission_report (relayed, not direct)
```

10. Implementation Plan (towards Phase 2)

1. Replace deterministic sweep with frontier exploration driven by the Writer's lidar plus an onboard occupancy grid, so coverage generalises beyond a hand-authored waypoint list to an unknown floor plan.
2. Real beacon hardware: pack the Beacon schema into a LoRa payload; replace the Gazebo-spawn deposition with an actual ejection mechanism on the physical drone.
3. Real Outside Network radio bridge: replace the /command_post topics with rosbridge plus MQTT over the intended long-range link, keeping outside_network_node's RECEIVE/TRANSLATE/CARRY/BRIEF logic unchanged.
4. SLAM-based localisation for both robots in place of ground-truth odometry, so the private robot coordinates the challenge refers to are genuinely estimated, not read from the simulator.
5. Physical prototype: a small quadrotor (Writer) and a differential- drive ground rover (Executor) carrying the same ROS2 stack, swapping Gazebo's plugins for real motor drivers and sensor nodes without touching living_map_system.

11. Failure Cases

1. Writer never reaches the entrance (battery exhaustion, physical loss). Beacons already dropped remain in the world and keep broadcasting - but the Writer's log of everything it saw never reaches the Outside Network, since handoff currently happens only once, at the entrance. Mitigation (planned, not yet implemented): periodic short-range broadcast attempts throughout exploration, not only at the end, so a partial log survives even total Writer loss.
2. Stale beacon with no ground truth to verify against. If the Executor reaches a beacon past the staleness threshold and its own sensors can not confirm the finding, it skips that target rather than acting on possibly outdated information, trading mission completeness for safety. Verified directly in testing (test_chain.py): a synthetic stale beacon with no corroborating sensor data was correctly skipped rather than acted on blindly.
3. GPS-anchor drift. The entrance GPS anchor is a single fixed reference point; if it is surveyed incorrectly, or the structure has shifted since survey, every translated beacon inherits that offset with no way to detect it. A redundant second anchor point would let the Outside Network cross-check and flag anchor error.
4. Host/deployment clock instability. Beacon aging and exploration time budgets depend on wall-clock time. During development, the simulation host exhibited a Hyper-V/WSL2 time-synchronisation fault causing multi-thousand-second clock jumps mid-run, which both stalled Gazebo's physics stepping and briefly corrupted a beacon-aging calculation in testing. Control-loop timing was hardened by switching to a monotonic clock (time.monotonic()), which is immune to such jumps; beacon aging intentionally keeps wall-clock time, since real beacon freshness must reflect real-world elapsed time - so a field deployment should independently monitor clock-sync health before trusting beacon ages at all.

12. Technologies Used

ROS2 Humble, Gazebo Classic 11 (gazebo_ros_pkgs), Python 3.10 (rclpy), standard ROS2 message

types (nav_msgs, sensor_msgs, geometry_msgs, gazebo_msgs, std_msgs),
colcon/ament_python/ament_cmake build tooling, and xacro for robot description templating.